

Faculté des Sciences et Techniques de Mohammedia

Département Informatique  
Année Universitaire 2025 – 2026

Rapport Académique

Projet de Recherche en Intelligence Artificielle

---

# Prédiction de la Vitesse du Trafic Routier par Apprentissage Profond sur Graphe

Temporal Graph Convolutional Network (T-GCN)

*Application au jeu de données PEMS07  
avec analyse d'explicabilité (XAI)  
et simulation de trafic sous SUMO*

---

## Encadrants

Pr. elBeggat Omar  
Pr. Bouziri adil  
Doctorant mohammed.laouadi

## Réalisé par

Haqiq Iliass  
Nassiri Zakariae  
Bensalah Hajar  
Merzaq Mouad

Mohammedia, Maroc

Juin 2026

## Table des matières

|           |                                                                           |           |
|-----------|---------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Le problème : prédire le trafic routier</b>                            | <b>2</b>  |
| 1.1       | Contexte et objectif . . . . .                                            | 2         |
| 1.2       | Pourquoi ce problème est difficile . . . . .                              | 2         |
| <b>2</b>  | <b>Les données et le prétraitement</b>                                    | <b>2</b>  |
| 2.1       | Description du jeu de données PEMS07 . . . . .                            | 2         |
| 2.2       | Normalisation Z-score . . . . .                                           | 2         |
| 2.3       | Fenêtres glissantes . . . . .                                             | 3         |
| <b>3</b>  | <b>Le graphe : encoder la géographie</b>                                  | <b>3</b>  |
| 3.1       | Pourquoi un graphe? . . . . .                                             | 3         |
| 3.2       | Construction de la matrice d'adjacence . . . . .                          | 3         |
| <b>4</b>  | <b>Le GCN : faire parler les voisins</b>                                  | <b>3</b>  |
| 4.1       | Rôle du GCN . . . . .                                                     | 3         |
| 4.2       | La formule . . . . .                                                      | 4         |
| 4.3       | Illustration concrète . . . . .                                           | 4         |
| <b>5</b>  | <b>Le GRU : la mémoire dans le temps</b>                                  | <b>4</b>  |
| 5.1       | Le problème que résout le GRU . . . . .                                   | 4         |
| 5.2       | Les équations du GRU . . . . .                                            | 4         |
| 5.3       | Rôle de chaque porte . . . . .                                            | 4         |
| <b>6</b>  | <b>Le T-GCN : comment GCN et GRU sont combinés</b>                        | <b>5</b>  |
| 6.1       | Les limites de chaque brique prise seule . . . . .                        | 5         |
| 6.2       | La fusion : remplacer $W \cdot [\cdot]$ par $\text{GCN}(\cdot)$ . . . . . | 5         |
| 6.3       | Ce que cela change concrètement . . . . .                                 | 5         |
| 6.4       | Une fusion intégrée, pas séquentielle . . . . .                           | 5         |
| 6.5       | Déroulement sur 12 pas de temps . . . . .                                 | 6         |
| <b>7</b>  | <b>L'entraînement : comment le modèle apprend</b>                         | <b>6</b>  |
| 7.1       | Vue d'ensemble de la boucle d'apprentissage . . . . .                     | 6         |
| 7.2       | La loss MAE . . . . .                                                     | 7         |
| 7.3       | Adam, gradient clipping et scheduler . . . . .                            | 7         |
| <b>8</b>  | <b>Résultats et interprétation</b>                                        | <b>7</b>  |
| 8.1       | Courbe d'apprentissage . . . . .                                          | 7         |
| 8.2       | Prédictions vs vérité terrain . . . . .                                   | 7         |
| 8.3       | Analyse spatiale de l'erreur . . . . .                                    | 8         |
| 8.4       | Matrice de confusion – états de trafic . . . . .                          | 8         |
| <b>9</b>  | <b>Explicabilité du modèle (XAI)</b>                                      | <b>9</b>  |
| 9.1       | Motivation . . . . .                                                      | 9         |
| 9.2       | Deux graphes très différents : distance vs corrélation . . . . .          | 9         |
| 9.3       | Ablation des voisins . . . . .                                            | 10        |
| 9.4       | Corrélation dynamique . . . . .                                           | 11        |
| <b>10</b> | <b>Simulation SUMO et jeu de données Casablanca</b>                       | <b>11</b> |
| 10.1      | Objectif . . . . .                                                        | 11        |
| 10.2      | Zone d'étude et cinq boulevards . . . . .                                 | 12        |
| 10.3      | Pipeline de simulation . . . . .                                          | 12        |
| 10.4      | Collecte des données et export . . . . .                                  | 12        |
| 10.5      | Mode démo et robustesse . . . . .                                         | 12        |
| <b>11</b> | <b>Conclusion</b>                                                         | <b>13</b> |

## 1 Le problème : prédire le trafic routier

### 1.1 Contexte et objectif

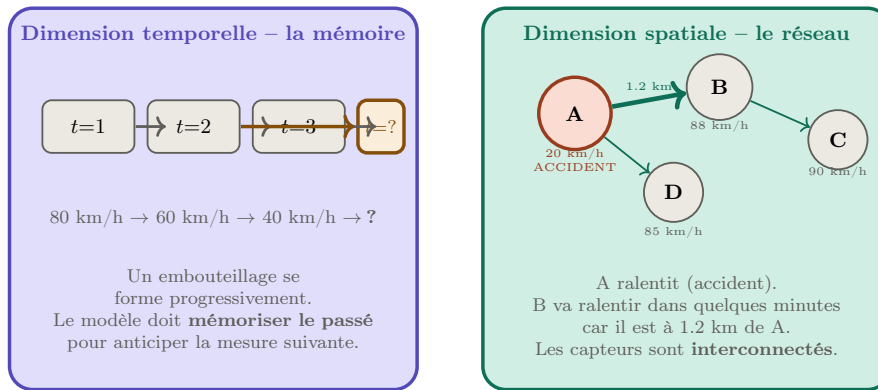
Le système californien PEMS (Performance Measurement System) repose sur des milliers de boucles magnétiques enterrées sous les autoroutes, chacune mesurant la vitesse moyenne des véhicules toutes les 5 minutes. Le problème que nous cherchons à résoudre peut se formuler très simplement :

#### Objectif

À partir des vitesses mesurées par 883 capteurs durant la **dernière heure** (12 pas de 5 minutes), prédire les vitesses de ces mêmes 883 capteurs pour la **prochaine heure** (12 pas futurs).

### 1.2 Pourquoi ce problème est difficile

Ce qui rend la tâche non triviale, c'est que le trafic possède deux dimensions de complexité qu'il faut traiter *en même temps* : une dimension temporelle (un ralentissement se construit progressivement, donc il faut une mémoire du passé) et une dimension spatiale (un incident sur un capteur se propage à ses voisins géographiques en quelques minutes). La figure 1 résume cette double contrainte.



**FIGURE 1** – Les deux dimensions de complexité du trafic routier. **Gauche** : dépendance temporelle – la séquence de vitesses passées permet d'anticiper la suivante. **Droite** : dépendance spatiale – un incident sur A se propage aux capteurs voisins B, C, D en quelques minutes. Ces deux dimensions doivent être capturées *simultanément*, ce qui motive l'architecture hybride présentée dans ce rapport.

C'est précisément cette complémentarité que le **T-GCN** exploite : un bloc **GCN** gère la dimension spatiale, un bloc **GRU** gère la dimension temporelle, et – comme nous le verrons en partie 6 – ces deux blocs ne sont pas simplement mis l'un après l'autre, ils sont imbriqués l'un dans l'autre.

## 2 Les données et le prétraitement

### 2.1 Description du jeu de données PEMS07

Le tableau 1 résume les caractéristiques principales du jeu de données utilisé.

**TABLE 1** – Caractéristiques du jeu de données PEMS07

| Propriété                   | Valeur         | Description                        |
|-----------------------------|----------------|------------------------------------|
| Capteurs ( $N$ )            | 883            | Boucles magnétiques, autoroutes CA |
| Résolution temporelle       | 5 minutes      | Un relevé toutes les 5 min         |
| Fenêtre d'entrée $T_h$      | 12 pas = 1 h   | Historique fourni au modèle        |
| Horizon de prédiction $T_p$ | 12 pas = 1 h   | Ce que le modèle doit prédire      |
| Grandeur mesurée            | Vitesse (km/h) | Vitesse moyenne par intervalle     |
| Arêtes du graphe            | 866 arêtes     | Distances entre capteurs voisins   |
| Taille train/val/test       | 60/20/20%      | Division temporelle chronologique  |

### 2.2 Normalisation Z-score

Les vitesses brutes varient entre 0 et plus de 100 km/h. Présenter directement ces valeurs au réseau de neurones rendrait l'apprentissage instable (les gradients seraient mal calibrés selon les capteurs). On applique donc une normalisation Z-score :

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} \quad (1)$$

où  $\mu$  et  $\sigma$  sont la moyenne et l'écart-type calculés *uniquement* sur l'ensemble d'entraînement. Après cette opération, les valeurs sont centrées autour de 0, typiquement entre  $-3$  et  $+3$ .

**Règle importante :**  $\mu$  et  $\sigma$  doivent être estimés sur le train uniquement. Les calculer sur l'ensemble complet laisserait des informations du futur “fuir” dans la normalisation – un biais classique qui donnerait des résultats artificiellement optimistes au moment de l'évaluation.

### 2.3 Fenêtres glissantes

Chaque exemple d'entraînement suit le schéma “12 derniers pas connus  $\rightarrow$  prédire les 12 pas suivants”. On fait glisser cette fenêtre d'un pas à chaque fois, ce qui multiplie le nombre d'exemples disponibles. Les tenseurs résultants ont les dimensions suivantes :

$$X \in \mathbb{R}^{N_{\text{samples}} \times 12 \times 883 \times 1} \quad (\text{entrée : 12 pas, 883 capteurs, 1 feature}) \quad (2)$$

$$Y \in \mathbb{R}^{N_{\text{samples}} \times 883 \times 12} \quad (\text{cible : 883 capteurs, 12 pas futurs}) \quad (3)$$

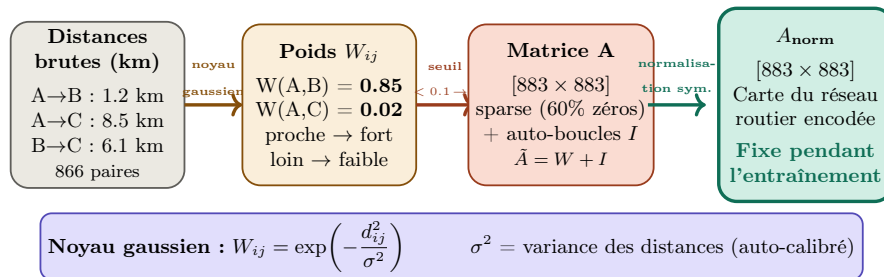
## 3 Le graphe : encoder la géographie

### 3.1 Pourquoi un graphe ?

Sans information de graphe, le modèle traiterait chaque capteur comme une île isolée : il ne pourrait jamais apprendre qu'un accident sur le capteur A va, en quelques minutes, ralentir le capteur B situé un kilomètre plus loin. Le graphe encode donc explicitement **qui est voisin de qui** sur le réseau autoroutier, et avec quelle “force” d'influence.

### 3.2 Construction de la matrice d'adjacence

La figure 2 décrit les quatre étapes qui transforment des distances brutes (en kilomètres) en une matrice  $A_{\text{norm}}$  exploitable par le réseau.



**FIGURE 2** – Les quatre étapes de construction de  $A_{\text{norm}}$ . **1.** Distances brutes en km. **2.** Conversion en poids via le noyau gaussien. **3.** Seuillage des liens faibles et ajout d'auto-boucles. **4.** Normalisation symétrique pour équilibrer l'influence de chaque capteur.

### Le noyau gaussien

$$W_{ij} = \exp\left(-\frac{d_{ij}^2}{\sigma^2}\right) \quad (4)$$

Concrètement : à distance nulle,  $W = 1.0$  (auto-boucle) ; à 1 km,  $W \approx 0.85$  (voisins proches, fortement couplés) ; à 8 km,  $W \approx 0.01$ , ce qui revient à supprimer le lien après seuillage.

### Pourquoi normaliser ?

Un capteur situé à un carrefour (10 voisins) recevrait, sans correction, bien plus d'information qu'un capteur en ligne droite (2 voisins). La normalisation symétrique  $\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$  divise chaque poids par  $\sqrt{k_i \cdot k_j}$  (où  $k$  est le nombre de voisins), ce qui équilibre l'influence reçue par chaque capteur, quel que soit son degré de connexion.

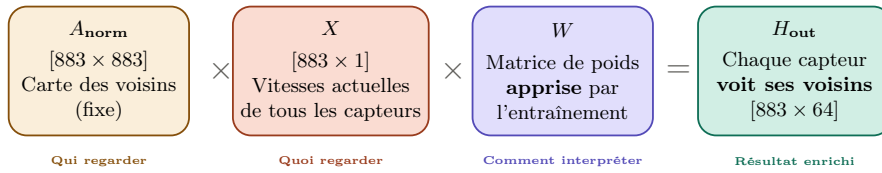
## 4 Le GCN : faire parler les voisins

### 4.1 Rôle du GCN

Le GCN (Graph Convolutional Network) permet à chaque capteur d'**agréger l'information de ses voisins**, pondérée par la force du lien géographique, avant de calculer sa propre représentation interne. Sans ce mécanisme, chaque capteur resterait aveugle à ce qui se passe autour de lui.

## 4.2 La formule

$$H_{\text{sortie}} = \text{ReLU}(A_{\text{norm}} \times X \times W) \quad (5)$$

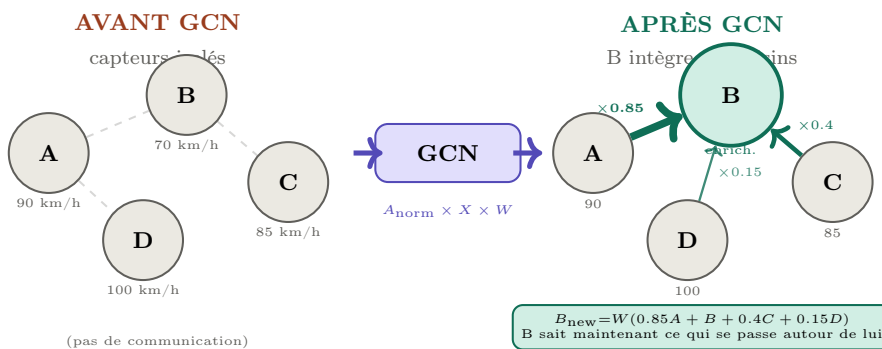


**FIGURE 3** – Décomposition de la formule GCN.  $A_{\text{norm}}$  choisit les voisins et leur poids,  $X$  contient les vitesses actuelles,  $W$  est la transformation apprise, et  $H_{\text{out}}$  est la représentation enrichie de chaque capteur.

## 4.3 Illustration concrète

Pour un capteur B ayant les voisins A (lien 0.85), C (lien 0.4) et D (lien 0.15), le GCN calcule :

$$B_{\text{nouveau}} = \text{ReLU}(W \times (0.85 V_A + 1.0 V_B + 0.40 V_C + 0.15 V_D)) \quad (6)$$



**FIGURE 4** – Avant et après une convolution sur graphe. Avant : chaque capteur ne connaît que sa propre vitesse. Après : B calcule une moyenne pondérée de ses voisins, l'épaisseur des flèches représentant la force du lien.

**Analogie** : c'est comme si chaque employé recevait, avant de prendre une décision, une synthèse des avis de ses collègues les plus proches – plus le collègue est proche, plus son avis compte dans la synthèse.

## 5 Le GRU : la mémoire dans le temps

### 5.1 Le problème que résout le GRU

Le GCN seul est **aveugle au temps** : il repart de zéro à chaque instant et ne sait pas qu'une vitesse a chuté progressivement sur les 12 derniers pas. Le GRU (Gated Recurrent Unit) maintient un **état caché**  $H_t \in \mathbb{R}^{N \times d_h}$  qui résume l'historique récent de chaque capteur, et décide à chaque instant quoi conserver et quoi oublier.

### 5.2 Les équations du GRU

$$u_t = \sigma(W_u \cdot [X_t \| H_{\text{prev}}]) \quad (\text{porte update}) \quad (7)$$

$$r_t = \sigma(W_r \cdot [X_t \| H_{\text{prev}}]) \quad (\text{porte reset}) \quad (8)$$

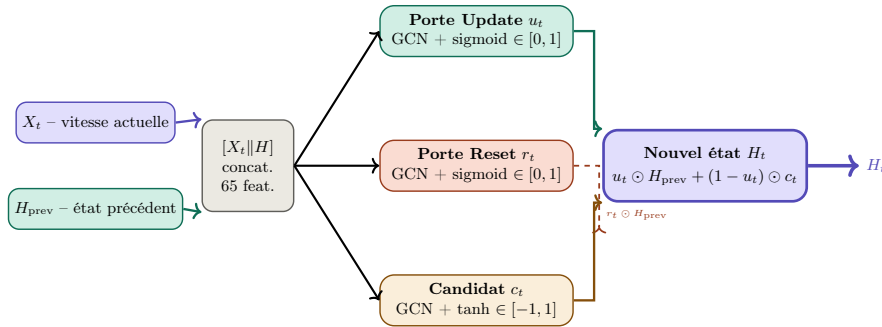
$$c_t = \tanh(W_c \cdot [X_t \| r_t \odot H_{\text{prev}}]) \quad (\text{candidat}) \quad (9)$$

$$H_t = u_t \odot H_{\text{prev}} + (1 - u_t) \odot c_t \quad (\text{nouvel état}) \quad (10)$$

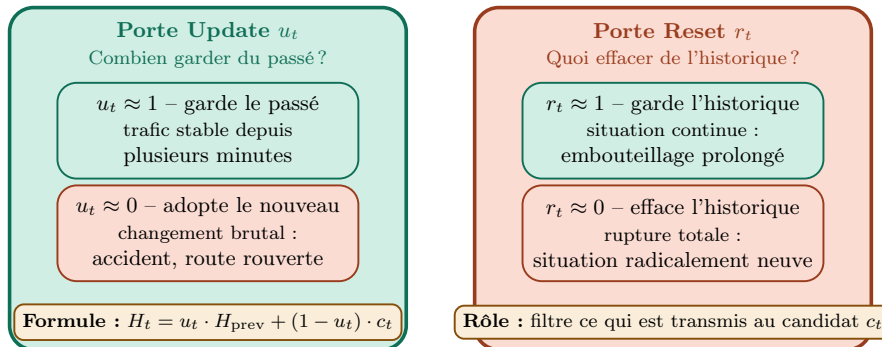
La figure 5 illustre comment ces trois calculs (reset, update, candidat) s'articulent à un instant  $t$  pour produire le nouvel état  $H_t$ .

### 5.3 Rôle de chaque porte

**Analogie – le carnet de route** :  $H_{\text{prev}}$  représente les notes passées du conducteur,  $X_t$  ce qu'il observe maintenant par la fenêtre. La porte reset se demande "mes notes sont-elles encore utiles?", la porte update "dans quelle proportion les mettre à jour?".  $H_t$  est le carnet mis à jour, mélange de l'ancien et du nouveau.



**FIGURE 5** – Flux d’une cellule GRU à un instant  $t$ .  $X_t$  et  $H_{\text{prev}}$  sont concaténés, puis trois calculs sont menés en parallèle : la porte reset filtre l’historique, la porte update choisit le ratio passé/nouveau, et le candidat propose le nouvel état. Le résultat  $H_t$  se propage au pas suivant.



**FIGURE 6** – Les deux portes du GRU. **Gauche** – porte update  $u_t$  : proportion de l’ancien état à conserver. **Droite** – porte reset  $r_t$  : pertinence de l’historique pour le nouvel état candidat.

## 6 Le T-GCN : comment GCN et GRU sont combinés

### 6.1 Les limites de chaque brique prise seule

Les équations (7) et (8) du GRU classique utilisent une simple multiplication matricielle  $W \cdot [X_t \| H_{\text{prev}}]$  : chaque capteur est traité de façon totalement indépendante, sans aucune notion de géographie. À l’instant  $t$ , quand le GRU décide “quoi garder, quoi oublier”, il ignore que le capteur B se trouve à 1.2 km du capteur A qui vient de ralentir. Inversement, le GCN seul applique une convolution sur le graphe à un instant précis, mais n’a aucune mémoire : il ne peut pas détecter qu’une vitesse baisse progressivement depuis 12 pas.

### 6.2 La fusion : remplacer $W \cdot [\cdot]$ par $\text{GCN}(\cdot)$

L’idée centrale du T-GCN est d’une simplicité trompeuse : **remplacer chaque multiplication matricielle des portes du GRU par une convolution de graphe**. C’est la seule modification apportée au GRU, mais elle suffit à injecter la topologie routière au cœur même du mécanisme de mémoire.

#### La substitution clé du T-GCN

$$\begin{aligned} \text{GRU classique : } u_t &= \sigma(\mathbf{W}_u \cdot [\mathbf{X}_t \| \mathbf{H}_{\text{prev}}]) \\ \text{T-GCN : } u_t &= \sigma(\text{GCN}([\mathbf{X}_t \| \mathbf{H}_{\text{prev}}])) \end{aligned}$$

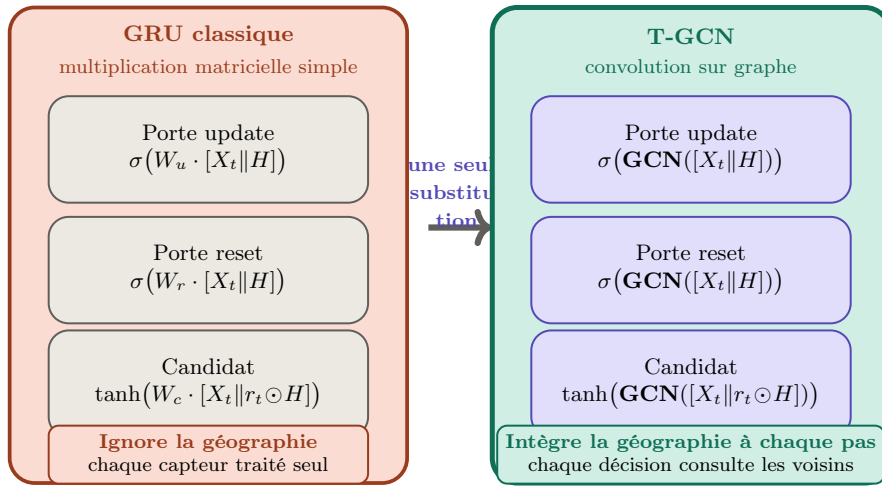
La même substitution s’applique à  $r_t$  et  $c_t$ . Le GCN interne utilise toujours la matrice fixe  $A_{\text{norm}}$  pour interroger les voisins à chaque calcul.

### 6.3 Ce que cela change concrètement

Le tableau 2 reprend l’exemple d’un accident sur le capteur A au pas  $t = 8$ , et compare le comportement du GRU classique à celui du T-GCN pour le capteur voisin B.

### 6.4 Une fusion intégrée, pas séquentielle

On aurait pu imaginer un schéma séquentiel : d’abord un GCN, puis un GRU classique sur le résultat. Mais dans le T-GCN, la fusion est **intime** : la mise à jour de la mémoire  $H_t$  tient compte des voisins *au moment même* où elle décide quoi oublier et quoi retenir. Ce n’est pas “regarder les voisins, puis mémoriser”, mais “mémoriser en regardant les voisins en même temps” – c’est cette nuance qui distingue véritablement le T-GCN d’une simple concaténation GCN+GRU.



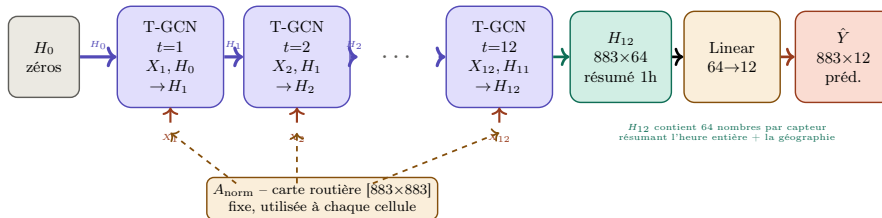
**FIGURE 7** – Comparaison GRU classique vs T-GCN. Les trois portes (update, reset, candidat) remplacent chacune leur multiplication matricielle par un GCN : chaque décision “retenir / oublier” consulte automatiquement les voisins géographiques via  $A_{\text{norm}}$ .

**TABLE 2** – Comparaison GRU classique vs T-GCN face à un accident

|               | GRU classique                         | T-GCN                                  |
|---------------|---------------------------------------|----------------------------------------|
| Ce que voit B | Uniquement $B = 88$ km/h              | Via le GCN : $A = 20$ km/h (lien 0.85) |
| Porte update  | Calcule sans savoir qu’A est en crise | Calcule en sachant qu’A ralentit       |
| Décision      | Garde l’historique fluide             | Détecte la rupture, ajuste $u_t$       |
| Prédiction    | B restera fluide (erreur)             | B va bientôt ralentir (correct)        |

## 6.5 Déroulement sur 12 pas de temps

La figure 8 montre comment l’état caché évolue pas à pas sur la fenêtre d’entrée, jusqu’à produire les 12 prédictions futures.



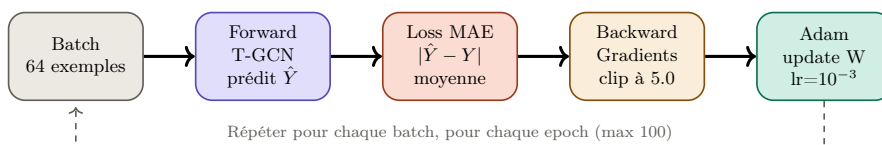
**FIGURE 8** – Déroulement du T-GCN sur les 12 pas d’entrée. L’état caché  $H_t$  s’enrichit à chaque pas en intégrant l’information temporelle (GRU) et spatiale (GCN interne),  $A_{\text{norm}}$  étant utilisée à chaque cellule.  $H_{12}$  ( $883 \times 64$ ) résume l’heure écoulée et est projeté vers 12 prédictions futures par une couche linéaire.

À la fin de la boucle, chaque capteur dispose de 64 nombres résumant tout ce qu’il faut savoir sur lui-même et ses voisins durant la dernière heure. Une simple couche linéaire transforme ces 64 nombres en 12 prédictions futures, soit une heure de prévision à venir.

## 7 L’entraînement : comment le modèle apprend

### 7.1 Vue d’ensemble de la boucle d’apprentissage

Au départ, les  $\approx 172\,000$  poids du T-GCN sont initialisés aléatoirement : le modèle prédit n’importe quoi. L’entraînement ajuste progressivement ces poids pour minimiser l’erreur de prédiction, selon la boucle représentée en figure 9.



**FIGURE 9** – Boucle d’entraînement complète. Chaque itération traite 64 exemples (forward), calcule l’erreur MAE, propage les gradients (backward) avec clipping à 5.0, et met à jour les poids via Adam.

## 7.2 La loss MAE

La fonction de perte, calculée dans l’espace normalisé, est la MAE (Mean Absolute Error) :

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N \cdot T_p} \sum_{i=1}^N \sum_{t=1}^{T_p} |\hat{y}_i^t - y_i^t| \quad (11)$$

Si le modèle prédit 75 km/h pour une vraie valeur de 80 km/h, l’écart est de 5 km/h ; on moyenne sur les 883 capteurs et les 12 pas futurs. La MAE est préférée à la MSE car elle pénalise les erreurs de façon proportionnelle, sans sur-amplifier les déviations ponctuelles extrêmes (accidents rares).

## 7.3 Adam, gradient clipping et scheduler

**Adam** (Adaptive Moment Estimation) adapte le pas d’apprentissage poids par poids, en maintenant une moyenne mobile des gradients ( $m_t$ , la “direction”) et une moyenne mobile de leurs carrés ( $v_t$ , la “variance”). Un poids stable reçoit un grand pas, un poids qui oscille reçoit un petit pas, ce qui accélère et stabilise la convergence par rapport à une descente de gradient classique.

**Pourquoi le gradient clipping ?** Le T-GCN boucle 12 fois sur sa cellule récurrente : à chaque itération, le gradient est multiplié par les poids internes. Si ces poids dépassent légèrement 1, après 12 multiplications le gradient explose (par exemple  $1.5^{12} \approx 129$ ). On limite donc la norme du gradient à  $\tau = 5.0$  sans changer sa direction :  $g \leftarrow \frac{\tau}{\|g\|_2} \cdot g$ .

Le scheduler **ReduceLROnPlateau** divise le taux d’apprentissage par 2 si la loss de validation stagne pendant 5 epochs — comme on ralentit sa recherche quand on se rapproche de la zone probable d’un objet perdu. L’**early stopping** arrête l’entraînement après 15 epochs sans amélioration et restaure le meilleur checkpoint, ce qui évite que le modèle ne mémorise les données d’entraînement au lieu de généraliser. Le tableau 3 résume tous les hyperparamètres.

TABLE 3 – Hyperparamètres de l’entraînement

| Paramètre               | Valeur    | Rôle                                 |
|-------------------------|-----------|--------------------------------------|
| Learning rate initial   | $10^{-3}$ | Taille du pas d’Adam                 |
| Weight decay            | $10^{-5}$ | Régularisation L2                    |
| Batch size              | 64        | Exemples par itération               |
| Gradient clipping       | 5.0       | Stabilité des réseaux récurrents     |
| Patience early stop     | 15 epochs | Détection du surapprentissage        |
| LR scheduler factor     | 0.5       | Division du LR si stagnation         |
| LR scheduler patience   | 5 epochs  | Seuil avant de diviser le LR         |
| Max epochs              | 100       | Limite supérieure d’entraînement     |
| Hidden dim $d_h$        | 64        | Taille de l’état caché par capteur   |
| Seuil graphe $\epsilon$ | 0.1       | $\approx 60\%$ des arêtes conservées |

## 8 Résultats et interprétation

### 8.1 Courbe d’apprentissage

La courbe se lit en deux temps. D’abord une **chute brutale** (epochs 1–5), de  $\approx 0.34$  à  $\approx 0.24$  : le modèle capte immédiatement les cycles jour/nuit, qui dominent les données et sont faciles à apprendre car très répétitifs. Vient ensuite une **convergence lente** (epochs 5–100), jusqu’à  $\approx 0.20$ , correspondant à l’apprentissage de motifs plus fins – différences lundi/samedi, conditions météo, événements ponctuels – qui demandent beaucoup plus d’ajustements sur les 172 000 poids. Les courbes de train et de validation restent proches tout au long de l’entraînement, on n’observe **pas de surapprentissage** ; le fait que la loss continue de baisser légèrement en fin d’entraînement suggère même qu’un entraînement plus long pourrait encore apporter un gain marginal.

### 8.2 Prédictions vs vérité terrain

Les **cycles journaliers** sont reproduits avec une grande fidélité (crêtes la nuit, creux aux heures de pointe), ce qui traduit directement la capacité du GRU à mémoriser la périodicité journalière. Quelques pics extrêmes (350–550 km/h, probablement des artefacts de mesure) sont **lissés** par le modèle : c’est un comportement attendu, puisqu’il a appris la dynamique normale du trafic et non ces anomalies rares. Lors des **ruptures brutales** (chute soudaine de vitesse), la prédiction suit rapidement la réalité – c’est l’effet de la porte reset, qui efface l’historique de trafic fluide pour adopter la nouvelle situation, illustrant concrètement l’intérêt de la fusion GCN+GRU. Enfin,

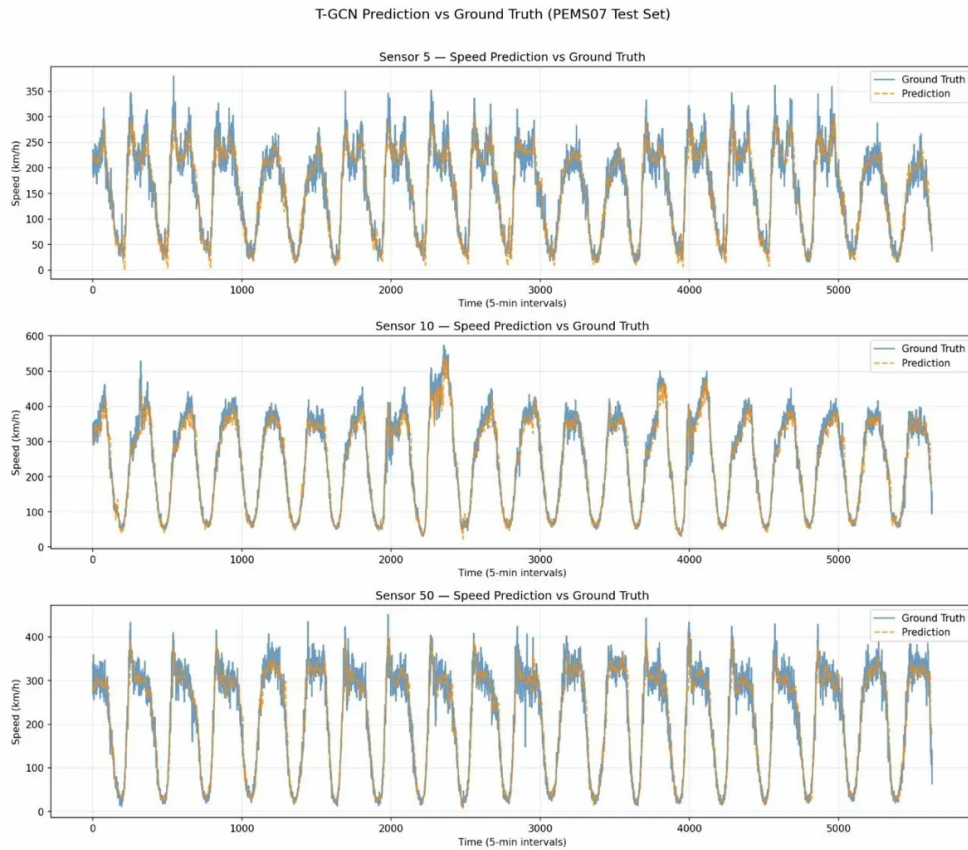


FIGURE 10 – Courbes de perte MAE (unités normalisées) sur 100 epochs. Bleu : loss d’entraînement. Orange pointillé : loss de validation.

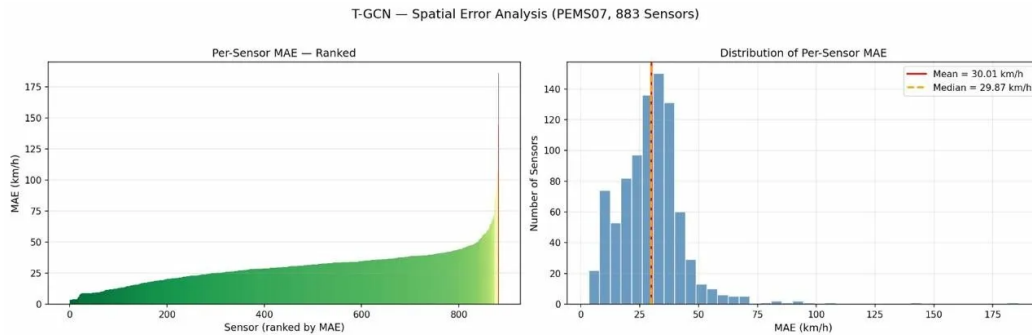


FIGURE 11 – Prédictions du T-GCN (orange pointillé) vs vitesses réelles (bleu) pour les capteurs 5, 10 et 50 sur l’ensemble de test. Axe  $x$  : intervalles de 5 minutes.

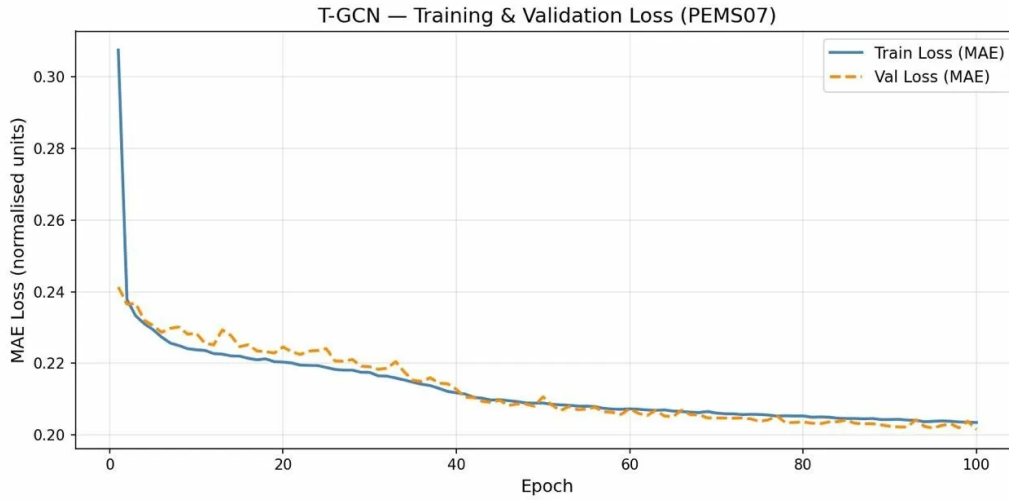
on observe une **variabilité inter-capteurs** : le capteur 50 (route droite, trafic homogène) est mieux prédit que le capteur 5 (carrefour plus complexe).

### 8.3 Analyse spatiale de l’erreur

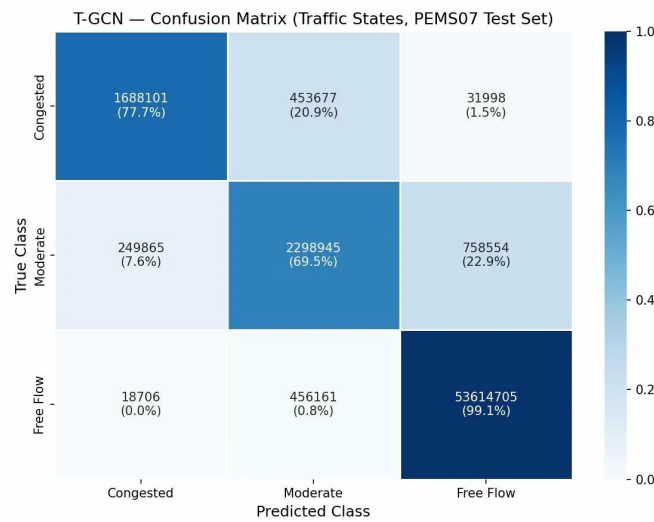
La quasi-égalité entre moyenne et médiane ( $\approx 30$  km/h) montre une performance homogène : environ 700 capteurs sur 883 ont une MAE comprise entre 10 et 45 km/h. Une petite minorité (40 à 50 capteurs,  $\approx 5\%$ ) dépasse 75 km/h de MAE – probablement des emplacements atypiques (carrefours complexes, zones de travaux, entrées/sorties d’autoroute) où le trafic est plus erratique. Sur la courbe triée, ces capteurs forment un **coude** net après le rang  $\approx 800$  : il s’agit donc d’une minorité structurellement difficile, distincte du reste. Pour un déploiement réel, ces capteurs mériteraient un traitement spécifique (modèle local, ou features supplémentaires comme le type de route).

### 8.4 Matrice de confusion – états de trafic

Le trafic **fluide** est identifié de façon quasi parfaite (99.1%), car cette classe correspond à des vitesses élevées, stables et régulières. Le trafic **congestionné** est correct dans 77.7% des cas : la majorité des erreurs basculent vers “modéré”, ce qui s’explique par le caractère arbitraire du seuil à 30 km/h (28 vs 32 km/h est une frontière difficile à trancher) ; seuls 1.5% des cas congestionnés sont classés “fluide”, ce qui est le type d’erreur le plus dangereux pour un système d’aide à la conduite. Le trafic **modéré** est la classe la plus délicate (69.5%), car elle correspond justement à la zone de transition entre les deux autres états. Le résultat le plus important reste



**FIGURE 12** – Analyse de l’erreur MAE par capteur. **Gauche** : MAE des 883 capteurs triée par ordre croissant. **Droite** : distribution des MAE, moyenne  $\approx 30.01$  km/h (rouge) et médiane  $\approx 29.87$  km/h (orange).



**FIGURE 13** – Matrice de confusion pour la classification des états de trafic (seuils : congestionné  $\leq 30$  km/h, modéré 30–60 km/h, fluide  $> 60$  km/h). Pourcentages normalisés par ligne.

l’absence de confusion grave entre les deux extrêmes (congestionné  $\leftrightarrow$  fluide,  $< 2\%$ ) : le modèle ne prédit jamais qu’une route est fluide alors qu’elle est totalement bouchée.

## 9 Explicabilité du modèle (XAI)

### 9.1 Motivation

Un modèle performant ne suffit pas pour un déploiement opérationnel : les gestionnaires de trafic ont besoin de comprendre *sur quels capteurs voisins* le modèle s’appuie pour anticiper la vitesse d’un nœud donné. Le module `explain.py` complète l’évaluation quantitative (`evaluate.py`) avec trois analyses interprétables, présentées ci-dessous.

### 9.2 Deux graphes très différents : distance vs corrélation

Le projet construit deux graphes d’adjacence : un graphe de **niveau 1** basé sur le noyau gaussien des distances physiques (celui utilisé jusqu’ici), et un graphe de **niveau 2** basé sur la corrélation de Pearson des vitesses sur le split d’entraînement. Le tableau 4 montre que ces deux graphes se recoupent très peu.

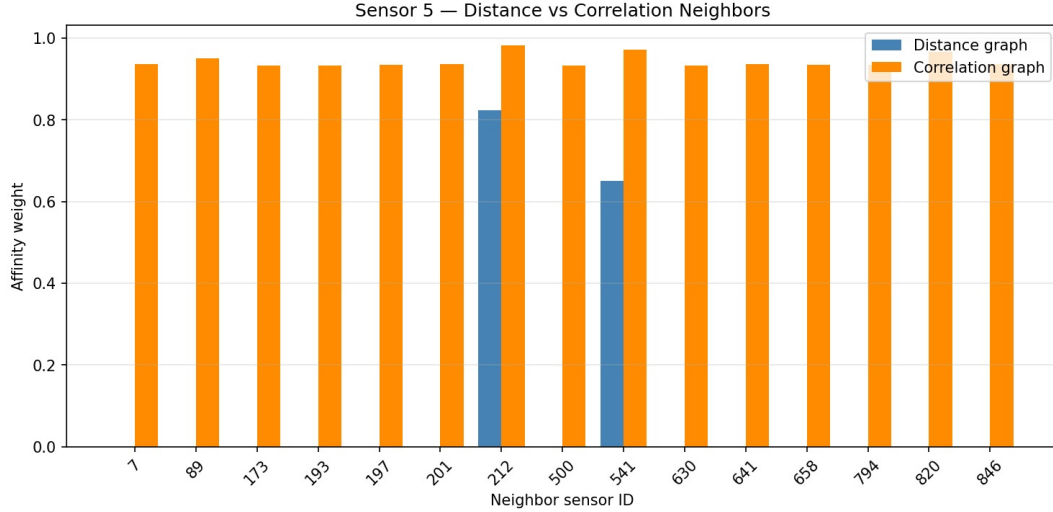
Ce contraste s’explique facilement : deux capteurs très éloignés peuvent avoir des profils de vitesse presque identiques s’ils connaissent les mêmes heures de pointe, ce que capture le graphe de corrélation – au prix d’une densité énorme et d’une perte d’interprétabilité géographique. Le graphe distance, lui, reste sparse et directement lisible sur une carte.

La figure 14 détaille ce contraste pour le capteur 5 : seuls deux voisins géographiques (212 et 541) apparaissent dans le graphe distance (barres bleues, affinités 0,82 et 0,65), tandis que le graphe de corrélation connecte ce même capteur à l’ensemble des 15 voisins affichés avec des affinités quasi uniformément supérieures à 0,93 (barres

**TABLE 4** – Recouvrement des arêtes (graphe distance vs corrélation, PEMS07)

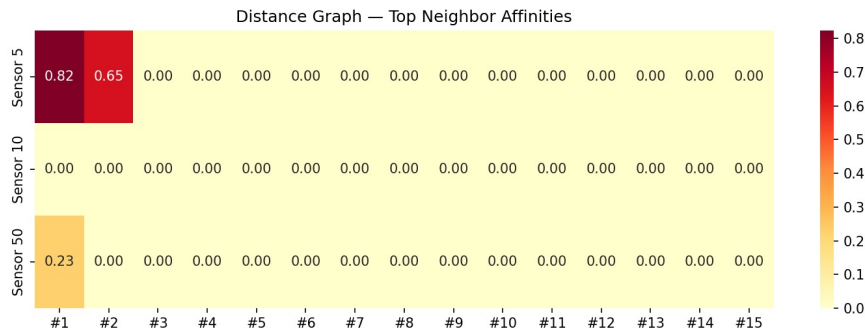
| Statistique                                | Valeur  |
|--------------------------------------------|---------|
| Arêtes (distance, non orientées)           | 790     |
| Arêtes (corrélation)                       | 382 304 |
| Arêtes communes                            | 788     |
| Indice de Jaccard global                   | 0,002   |
| Jaccard moyen (top-15 voisins par capteur) | 0,080   |

orange). Les deux graphes capturent donc des relations fondamentalement différentes : la proximité géographique d’un côté, la similarité de profil temporel de l’autre.



**FIGURE 14** – Comparaison des affinités distance et corrélation pour le capteur 5 et ses 15 voisins principaux (identifiants sur l’axe  $x$ ). **Barres bleues** : poids du graphe distance (noyau gaussien) – seuls les capteurs 212 (0,82) et 541 (0,65) sont présents, les autres étant à zéro après seuillage. **Barres orange** : poids de corrélation de Pearson – tous les voisins affichent une affinité élevée ( $> 0,93$ ), indépendamment de leur distance physique. Ce contraste justifie l’intérêt d’un graphe de corrélation dynamique en complément du graphe géographique fixe.

La figure 15 illustre, pour les capteurs 5, 10 et 50, les affinités avec leurs 15 principaux voisins dans le graphe distance (heatmap) : on y voit immédiatement que le capteur 5 a deux voisins forts (0,82 et 0,65), le capteur 50 un seul voisin modéré (0,23), et le capteur 10 **aucun** voisin significatif – il est donc isolé dans ce graphe géographique.



**FIGURE 15** – Affinités des 15 principaux voisins dans le graphe distance, pour les capteurs 5, 10 et 50 (heatmap). Le capteur 5 possède deux voisins géographiques actifs (0,82 et 0,65), le capteur 50 un seul (0,23), et le capteur 10 aucun – il est géographiquement isolé après seuillage.

### 9.3 Ablation des voisins

Pour mesurer concrètement l’influence d’un voisin  $j$  sur la prédiction d’un capteur cible  $s$ , on procède par **ablation d’arête** sur le graphe d’entraînement :

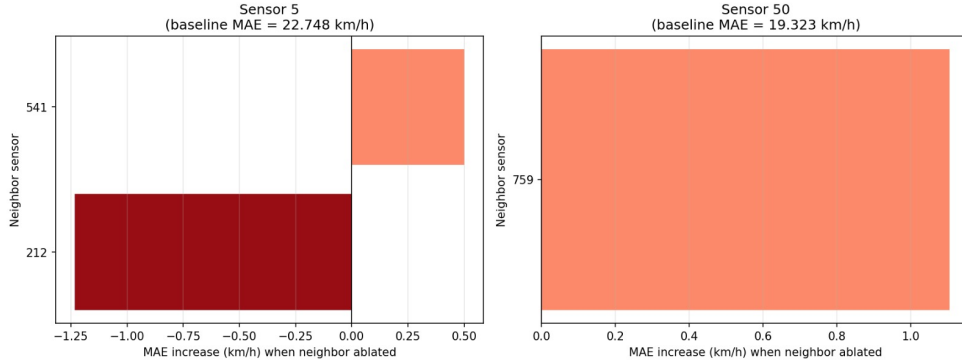
1. inférence sur 200 échantillons de test  $\Rightarrow$  MAE de référence  $\text{MAE}_0$  ;
2. pour chaque voisin  $j \in \mathcal{N}(s)$ , suppression de l’arête  $(s, j)$ , re-normalisation de  $\tilde{A}$ , nouvelle inférence ;
3. score d’influence  $\Delta_j = \text{MAE}_j - \text{MAE}_0$ .

Un  $\Delta_j$  élevé et positif signifie que retirer le voisin  $j$  **dégrade** la prédiction – le modèle en dépend fortement. Un  $\Delta_j$  négatif indique au contraire un voisin potentiellement redondant, voire bruyant. Le tableau 5 et la figure 16 présentent ces scores pour quelques capteurs.

**TABLE 5** – Résultats d’ablation (graphe distance, checkpoint epoch 2)

| Capteur | Voisins les plus influents (top-3)       | $\Delta$ MAE (km/h) |
|---------|------------------------------------------|---------------------|
| 5       | 541, 212                                 | +0,50 ; -1,23       |
| 10      | — (aucun voisin dans le graphe distance) | —                   |
| 50      | 759                                      | +1,11               |

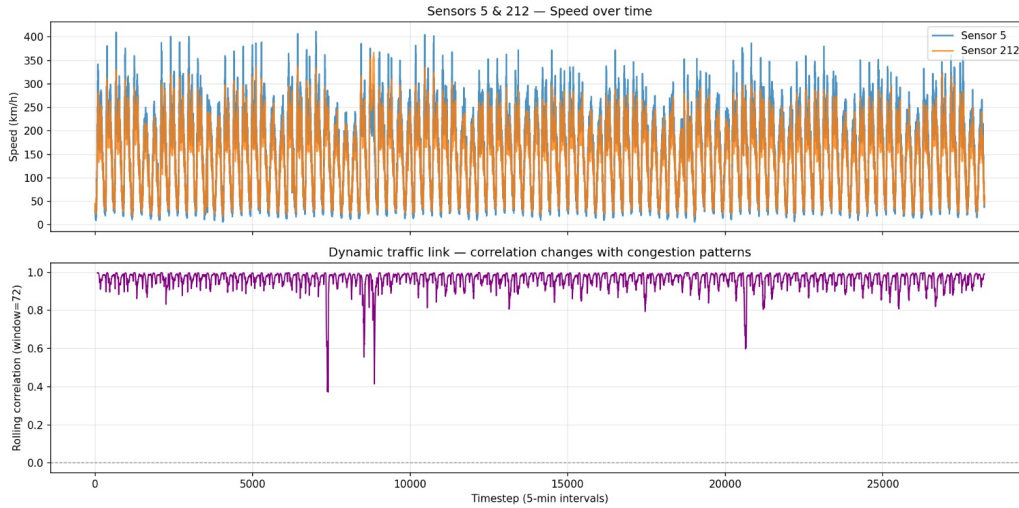
XAI – Neighbor Ablation (which neighbors matter most?)

**FIGURE 16** – Impact de l’ablation de chaque voisin sur la MAE de prédiction (capteurs 5 et 50). Une barre positive ( $\Delta > 0$ ) indique que supprimer ce voisin **dégrade** la prédiction ; une barre négative signifie que ce voisin est redondant ou bruité.

On notera que le capteur 10 n’a aucun voisin dans le graphe distance après seuillage – ce que confirme déjà la figure 15 : son comportement repose donc uniquement sur sa propre dynamique temporelle (via le GRU), ce qui constitue en soi un résultat interprétable, ce capteur étant “isolé” géographiquement au sens du graphe construit.

#### 9.4 Corrélation dynamique

En complément de l’analyse modèle-centrée précédente, on calcule une corrélation de Pearson *roulante* entre paires de capteurs (fenêtre  $\approx 1$ h, soit 72 pas de 5 min). Elle révèle comment les liens de trafic **évoluent dans le temps** : forte corrélation aux heures de pointe, affaiblissement en heures creuses. Ce constat justifie l’intérêt d’un graphe de corrélation dynamique en complément du graphe géographique fixe, qui par construction ne peut représenter que des relations stables.

**FIGURE 17** – Vitesses et corrélation roulante entre les capteurs 5 et 212 (voisin de corrélation le plus proche du capteur 5). La corrélation reste proche de 1,0 la majorité du temps, avec des chutes ponctuelles lors de ruptures de trafic, illustrant le caractère dynamique des dépendances spatiales.

**Commande :** `python explain.py` (depuis la racine TGCN/, après `train.py`).

## 10 Simulation SUMO et jeu de données Casablanca

### 10.1 Objectif

En complément du benchmark PEMS07 (883 capteurs, Californie), nous avons construit un **second jeu de données**, beaucoup plus petit mais entièrement maîtrisé : 5 boulevards du centre de Casablanca, simulés au niveau microscopique avec **Eclipse SUMO** à partir de données OpenStreetMap. L’objectif est double : disposer

d’un jeu de données “contrôlable” (on connaît la cause exacte de chaque variation de trafic, puisqu’on a simulé la demande), et vérifier que l’architecture T-GCN se généralise à un graphe beaucoup plus petit ( $N = 5$  au lieu de  $N = 883$ ).

## 10.2 Zone d’étude et cinq boulevards

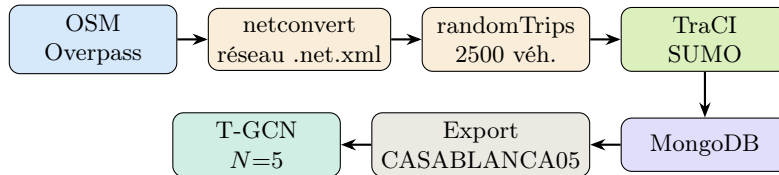
La zone couvre environ  $3 \times 3$  km autour du centre-ville ( $33.5731^\circ\text{N}$ ,  $7.5898^\circ\text{W}$ ). Cinq artères forment une grille simplifiée, résumée dans le tableau 6.

**TABLE 6** – Cinq boulevards modélisés (`simulation/config.py`)

| ID | Boulevard      | Orientation | Arêtes SUMO |
|----|----------------|-------------|-------------|
| 0  | Mohammed V     | Nord-Sud    | $\sim 185$  |
| 1  | Zerktouni      | Nord-Sud    | $\sim 168$  |
| 2  | Hassan II      | Est-Ouest   | $\sim 162$  |
| 3  | Avenue des FAR | Est-Ouest   | $\sim 162$  |
| 4  | Brahim Roudani | Connecteur  | $\sim 162$  |

## 10.3 Pipeline de simulation

Le script `simulation/run_pipeline.py` enchaîne six étapes, schématisées en figure 18 : récupération du réseau routier via Overpass (OSM), conversion en réseau SUMO via `netconvert`, génération d’une demande synthétique d’environ 2500 véhicules (`randomTrips`), exécution de la simulation pas à pas via l’API `TraCI`, stockage des séries temporelles dans `MongoDB`, puis export vers un format compatible avec `train.py -dataset casablanca`.



**FIGURE 18** – Pipeline complet : du réseau OSM à l’entraînement du T-GCN sur le graphe Casablanca à 5 nœuds.

## 10.4 Collecte des données et export

La simulation dure 3600 s (1 h) avec un pas d’échantillonnage de 60 s ; les 300 premières secondes servent de période de chauffe pour laisser le trafic se stabiliser. Le tableau 7 résume les paramètres principaux. À chaque minute simulée, TraCI agrège pour chaque boulevard la vitesse moyenne, le débit, la densité et le taux d’occupation, via un mapping automatique entre boulevards et arêtes SUMO basé sur les noms OSM (`generate_demand.py`).

**TABLE 7** – Paramètres principaux de la simulation

| Paramètre             | Valeur                       |
|-----------------------|------------------------------|
| Durée simulée         | 3600 s                       |
| Pas d’échantillonnage | 60 s                         |
| Période de chauffe    | 300 s                        |
| Véhicules générés     | $\sim 2500$                  |
| Split MongoDB         | 40% test / 60% entraînement  |
| Base MongoDB          | <code>tgcn_casablanca</code> |

Quatre collections MongoDB stockent respectivement les métadonnées (`simulation_runs`), les séries temporelles (`speed_timeseries`), les agrégats par boulevard (`boulevard_metrics`) et le découpage train/test (`dataset_splits`). Le module `export_tgcn.py` produit ensuite `data/CASABLANCA05.npz` (matrice de vitesses  $[T, 5]$ ) et `data/CASABLANCA05.csv` (distances géodésiques entre boulevards). Le même modèle T-GCN ( $d_h = 64$ , fenêtre 12/12) peut alors être ré-entraîné sur ces 5 nœuds, démontrant la **réutilisabilité directe** de l’architecture, indépendamment de la taille du graphe.

## 10.5 Mode démo et robustesse

Si SUMO n’est pas installé (variable `SUMO_HOME` absente), le pipeline bascule automatiquement en **mode démo** : des séries synthétiques réalistes sont générées à la place de la microsimulation, ce qui permet de tester toute la chaîne (MongoDB, export, `train.py -dataset casablanca`) sans dépendance externe. Les commandes principales sont :

— `python -m simulation.run_pipeline -demo` – test complet sans SUMO ;

- `python -m simulation.run_pipeline` – pipeline OSM + SUMO + MongoDB complet ;
- `python -m simulation.run_pipeline -step export` – export vers le format T-GCN ;
- `python train.py -dataset casablanca -graph distance` – entraînement sur le graphe Casablanca.

Le lien avec PEMS07 est direct : même format de données, même construction de graphe par noyau gaussien, même cellule T-GCN – seule la source des séries change (capteurs physiques réels vs simulation microscopique sur 5 artères).

## 11 Conclusion

**TABLE 8** – Synthèse des performances du T-GCN sur le test set PEMS07

| Aspect                            | Résultat               | Qualité    |
|-----------------------------------|------------------------|------------|
| MAE médiane (tous capteurs)       | $\approx 30$ km/h      | Correcte   |
| Convergence (courbe de loss)      | Stable, sans explosion | Bonne      |
| Surapprentissage (overfitting)    | Absent                 | Bonne      |
| Trafic fluide (classification)    | 99.1%                  | Excellente |
| Trafic congestionné               | 77.7%                  | Bonne      |
| Trafic modéré                     | 69.5%                  | Acceptable |
| Confusion extrêmes (cong./fluide) | < 2%                   | Excellente |

Le T-GCN propose une solution élégante au problème de prédiction spatio-temporelle : en remplaçant les multiplications matricielles du GRU par des convolutions de graphe, il intègre la topologie routière à *l'intérieur même* du mécanisme de mémoire, plutôt que de traiter les deux dimensions l'une après l'autre. Avec seulement  $\approx 172\,000$  paramètres, le modèle atteint de bonnes performances sur 883 capteurs, sans surapprentissage et avec une classification quasi parfaite des états de trafic fluide. L'analyse d'explicabilité montre que cette performance repose effectivement sur le graphe géographique (et non sur une simple mémorisation par capteur), tandis que l'extension SUMO/Casablanca confirme que l'architecture se transpose, sans modification, à un graphe routier de taille très différente. Les pistes d'amélioration les plus prometteuses restent le traitement spécifique des  $\sim 5\%$  de capteurs structurellement difficiles, et l'exploitation conjointe du graphe de corrélation dynamique mis en évidence en partie 9.

## Références

- [1] Zhao, L. *et al.* (2020). *T-GCN : A Temporal Graph Convolutional Network for Traffic Prediction*. IEEE Trans. Intelligent Transportation Systems, 21(9), 3848–3858.
- [2] Kipf, T. N., & Welling, M. (2017). *Semi-Supervised Classification with Graph Convolutional Networks*. ICLR 2017.
- [3] Cho, K. *et al.* (2014). *On the Properties of Neural Machine Translation*. arXiv :1409.1259.
- [4] Kingma, D. P., & Ba, J. (2015). *Adam : A Method for Stochastic Optimization*. ICLR 2015.
- [5] Caltrans (2023). *Performance Measurement System (PeMS)*. California Department of Transportation.